Klasse:	LF 5.5 Einfache Anwendungen in Python implementieren	BBS I Berufsbildende Schulen I des Landkreises Lüneburg
Datum:	**Thema: Operatoren**	Anika Ringe

Um in Verzeigungen, Schleifen usw. zwei oder mehr Werte miteinander zu vergleichen, werden relationale Operatoren benötigt. Diese heißen auch Vergleichsoperatoren und geben ein logisches Ergebnis “True“ oder “False“ aus.

Vergleichsoperatoren		
Operator	Bezeichnung	Beschreibung
a == b	gleich	Es wird überprüft, ob der Wert der variablen a dem Wert der Variablen b entspricht (gleich). In dem Fall, dass dies zutrifft, ist das Ergebnis der Operation true.
a != b	ungleich	Das Ergebnis der Operation ist true, wenn a ungleich b ist.
a > b	Größer als	Das Ergebnis der Operation ist true, wenn a größer als b ist.
a < b	Kleiner als	Das Ergebnis der Operation ist true, wenn a kleiner als b ist.
a >= b	Größer-gleich	Das Ergebnis der Operation ist true, wenn a größer oder gleich b ist.
a <= b	Kleiner-gleich	Das Ergebnis der Operation ist true, wenn a kleiner oder gleich b ist.

Boolesche Operatoren werden auch logische Operatoren genannt, weil sie der logischen Verknüpfung von Aussagen dienen.

Boolesche Operatoren		
Operator	Name	Bedeutung
not	Negation	Invertiert den Ausdruck (kehrt in um).
and	Und	Ergibt True, wenn beide Operanten True sind.
or	Oder	Ergibt True, wenn mindestens ein Operant True ist.

Wahrheitstabelle der einzelnen Operatoren							
x	**y**	**x and y**	**x or y**	**not x**	**not y**	**not (x and y)**	**not (x or y)**
True	**True**	True	True	False	False	False	False
True	**False**	False	True	False	True	True	False
False	**True**	False	True	True	False	True	False
False	**False**	False	False	True	True	True	True

Zusatzinformationen

<u>Bit-Operatoren</u>

Bit-Operatoren arbeiten auf der Bit-Ebene und können nur für ganzzahlige Typen verwendet werden. Hierbei wird jedes einzelne Bit miteinander verglichen und je nach Operator wird für dieses Bit ein Ergebnis erzeugt.

Bit-Operatoren		
Operator	Name	Bedeutung
~	Bitweise Negation	Invertiert den Ausdruck (kehrt in um).
&	Bitweise Und	Ergibt True, wenn beide Operanten True sind.

Quelle: westermann, Python

|	Bitweise Oder	Ergibt True, wenn mindestens ein Operant True ist.
^	Bitweise Exclusiv Oder (XOR)	Ergibt True, wenn genau ein Operant True ist
<<	Shift links	Verschiebt die Bits eines Wertes um eine bestimmte Anzahl nach links. Alle Bits auf der rechten Seite werden mit Nullen aufgefüllt.
>>	Shift rechts	Verschiebt die Bits eines Wertes um eine bestimmte Anzahl nach rechts. Alle Bits auf der linken Seite werden mit Nullen aufgefüllt.

Die Ergebnisse pro Operator, bezogen auf ein einzelnes Bit, sind in der nachfolgenden Tabelle noch einmal dargestellt.

x	**y**	**x & y**	**x |y**	**x ^ y**	**~x**	**~y**
1	**1**	1	1	0	0	0
1	**0**	0	1	1	0	1
0	**1**	0	1	1	1	0
0	**0**	0	0	0	1	1

<u>Identity-Operatoren</u>
Identity-Operatoren, auch Identitäts-Operatoren genannt, dienen dazu, Referenzen auf Objekte miteinander zu vergleichen.

Identity Operatoren		
Operator	Bedeutung	Beispiel
is	Gibt True zurück, wenn beide Werte auf dasselbe Objekt zeigen, ansonsten False.	x = 100 print(type(x) **is** int) # Ausgabe ist True
Is not	Gibt True zurück, wenn beide Werte nicht auf dasselbe Objekt zeigen, ansonten False.	x = 1000 pint(type(x) **is not** float) #Ausgabe ist True.

<u>Membership-Operatoren</u>
Mithilfe von Membership Operatoren lässt sich feststellen, ob ein bestimmtes Element in einer Wertesammlung vorhanden ist.

Membership-Operatoren		
Operator	Bedeutung	Beispiel
in	Gibt True zurück, wenn ein Wert in einer bestimmten Wertesammlung gefunden wurde. Ansonsten wird False zurückgegeben.	liste = [1,2,3,4] x = 2 print(x **in** liste) #Ausgabe True

Quelle: westermann, Python

not in	Gibt True zurück, wenn ein Wert in einer bestimmten Wertesammlung nicht gefunden wurde. Ansonsten wird False zurückgegeben.	liste = [1,2,3,4] x = 2 print(x **not in** liste) #Ausgabe false

Quelle: westermann, Python